CIA – 3

Student Attendance Academic Monitoring System

(MIS Web Application)

Submitted to

DR. HELEN JOSEPHINE V L

By

NIMISHA SAIKIA

2527333

MBA PROGRAMME

SCHOOL OF BUSINESS AND MANAGEMENT

CHRIST (DEEMED TO BE UNIVERSITY), BANGALORE

MBA 342B – Python for Data Analysis (PDA)

March 2026

INDEX

Sl. No.	Topic	Page No.
1.	**Problem Statement**	3
2.	**Objectives**	3
	2.1. Primary Objectives	-
	2.2 Secondary Objectives	-
3.	**System Design**	4
	3.1. Architecture	-
	3.2. ER Diagram	-
	3.3. File Structure	-
	3.4. Activity Diagram	-
4.	**Codes**	7
	4.1. database.py	-
	4.2. calculations.py	-
	4.3. app.py	-
5.	**Dataset**	13
	5.1. My SQL Schema	-
	5.2. Sample Data	-
6.	**Output**	15
7.	**Github Repository**	19
	7.1.Repository Structure	-
	7.2. Running the project locally	-
8.	**Conclusion**	20

1. Problem Statement

Academic institutions require students to maintain a minimum attendance of 85% per subject to remain eligible for semester examinations. The current process of tracking attendance is manual — faculty maintain spreadsheets or paper registers, and students have no real-time visibility into their standing until eligibility lists are published near the end of the trimester.

This creates three critical gaps: students discover ineligibility too late to recover, faculty spend significant administrative time on calculations, and approved absences (medical leave, extracurricular activities) are not factored into eligibility in a transparent or automated way.

This project — the Student Attendance Academic Monitoring System — is a web-based Management Information System (MIS) built using Python (Flask) and MySQL that addresses these gaps through two role-specific portals:

- **Student Portal:** real-time attendance dashboard, per-subject breakdown, leave claim simulation, and eligibility projection.
- **Teacher Portal:** mark present/absent for any student-subject combination, edit attendance counts, and manage class records.

2. Objectives

2.1. Primary Objectives

1. Build a web-based MIS with role-specific access for students and teachers, backed by a MySQL relational database.
2. Calculate overall and per-subject attendance percentages accurately, with correct handling of approved leave claims.
3. Display the exact number of consecutive classes a student must attend to reach the 85% threshold.
4. Allow students to simulate the effect of medical or extracurricular claims on their attendance standing before submitting them.
5. Enable teachers to mark attendance and edit records without spreadsheets, with changes reflected immediately.

2.2. Secondary Objectives

- Centralise all arithmetic in a single reusable module (calculations.py) for maintainability and testability.
- Use modular Flask routing where each function has a single, clear responsibility.
- Deliver a self-contained interface (no external stylesheet dependency) across five Jinja2 templates.
- Host the project on GitHub for version control and reproducibility.

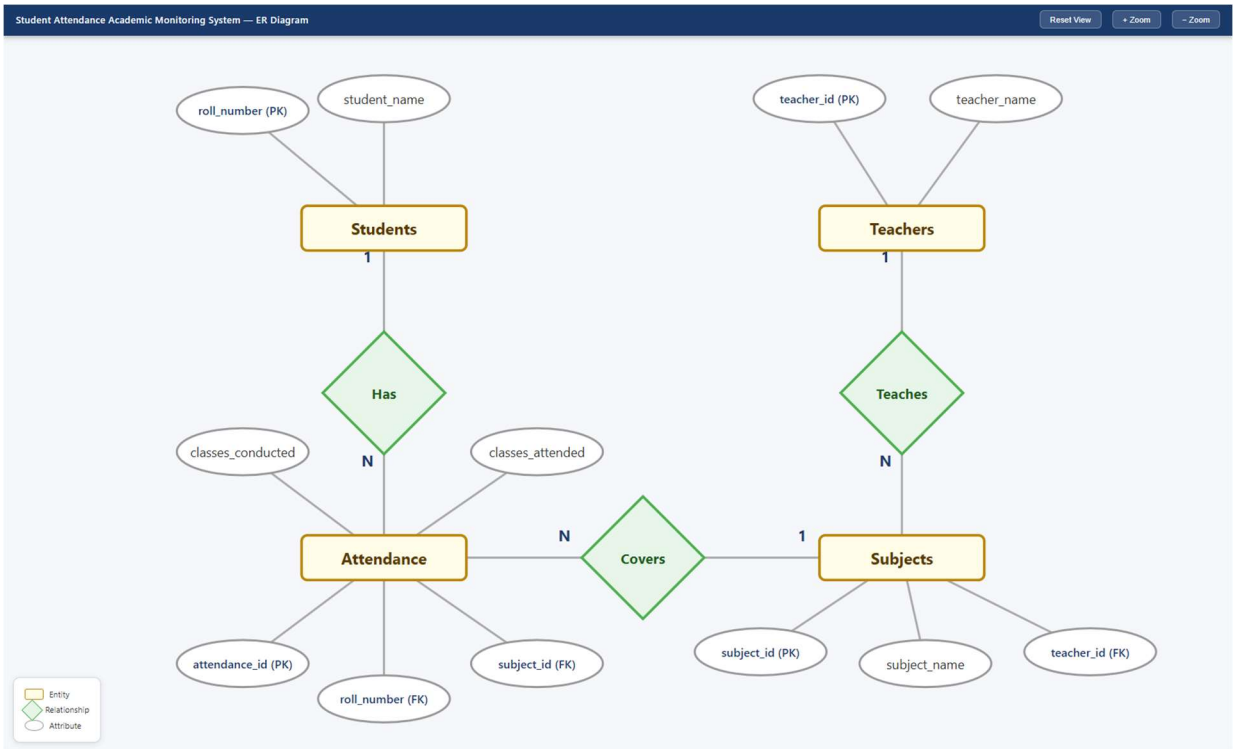
3. System Design

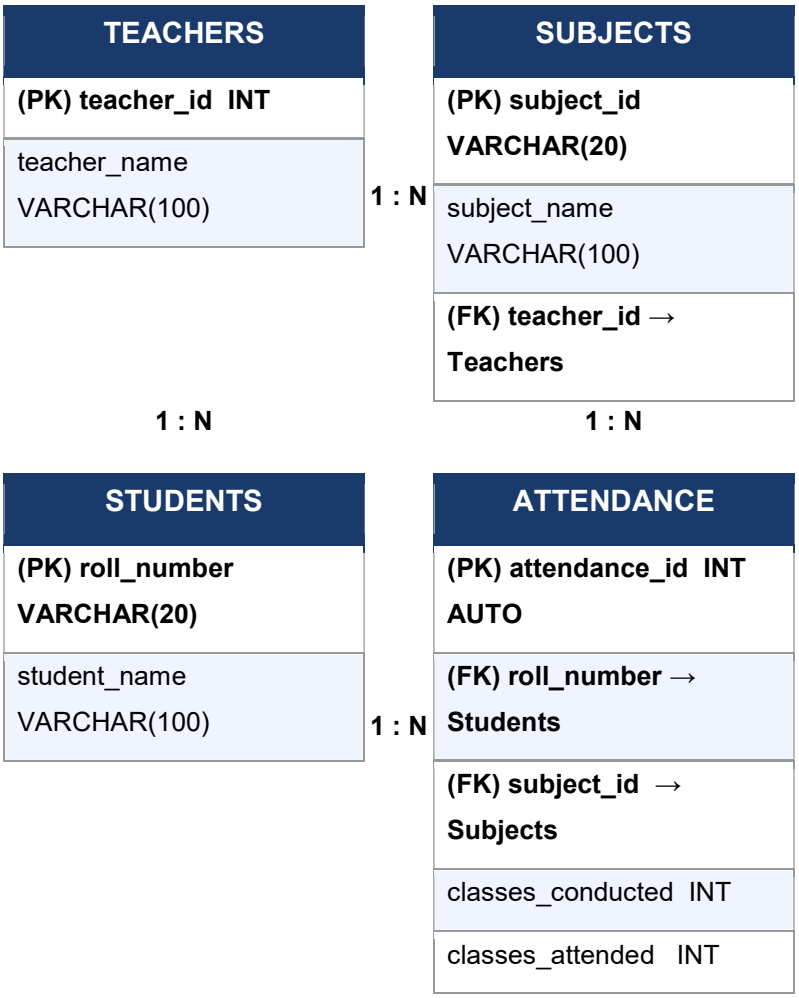
3.1. Architecture

Layer	Technology	Role
Model	MySQL + mysql-connector-python	Stores Students, Teachers, Subjects, Attendance
Controller	Python / Flask (app.py + calculations.py)	Route handling, business logic, metric computation
View	Jinja2 HTML templates	Student dashboard, teacher dashboard, login pages
Config	database.py	Centralised MySQL connection credentials

3.2. ER Diagram

The database has four entities. Attendance is the central fact table linking Students and Subjects, each of which has a 1:N relationship with it.





3.3. File Structure

```
Student-Attendance-Academic-Monitoring-System/
├── app.py                        # Flask routes & application logic
├── calculations.py               # Attendance metric computation
├── database.py                   # MySQL connection helper
├── requirements.txt              # Python dependencies
├── README.md
├── .gitignore
├── templates/
│   ├── index.html                # Role selector (Student / Teacher)
│   ├── student_login.html        # Student portal login
│   ├── teacher_login.html        # Teacher portal login
│   ├── student_dashboard.html    # Attendance dashboard with ring chart
│   └── teacher_dashboard.html    # Teacher attendance management
```

4. Activity Diagram

The diagram below shows the complete application flow from user entry through both portals, including all branching paths.

START	
User visits http://localhost:5000/	
index.html — Select Role	
Student	Teacher
— STUDENT PORTAL —	**— TEACHER PORTAL —**
student_login.html — Enter Roll Number	teacher_login.html — Enter Teacher ID
POST /student — Query Students table	POST /teacher — Fetch Attendance for teacher's subjects
[Not found] → Return 404 / [Found] → Fetch Attendance records	Render teacher_dashboard.html
JOIN Subjects + Teachers → build per-subject record list	[Mark Present] → /mark_present → attended+1, conducted+1
calculate_metrics() — compute %, hours, classes_needed	[Mark Absent] → /mark_absent → conducted+1 only
Render student_dashboard.html (ring chart, subject table, sim panel)	[Edit Attended] → /edit_attendance → SET classes_attended = value
Student submits Medical / Extra claims → POST /simulate_claim	[Edit Conducted] → /edit_conducted → SET classes_conducted = value
Re-run calculate_metrics(claims) → re-render (simulated = True)	All teacher actions commit to MySQL then redirect to /teacher
Student clicks Reset → POST /reset_dashboard → redirect 307 /student	
END	

4. CODE

4.1. database.py — MySQL Connection

Centralises database credentials. All modules call get_connection() — credentials are defined in one place.

```
import mysql.connector


def get_connection():
    return mysql.connector.connect(
        host     = "localhost",
        user     = "root",
        password = "Student@1484",
        database = "attendance_system"
    )
```

4.2. calculations.py — Metric Engine

All attendance arithmetic is centralised here. Claims (medical / extracurricular) correctly reduce the effective conducted denominator — not inflate the attendance numerator.

Formula — Classes Needed for 85%

Solve for x: (Attended + x) / (Effective Conducted + x) = 0.85 → x = (0.85 × EC − Attended) / 0.15

```
import math
CLASS_HOURS = 1.5


def calculate_metrics(attended, conducted, medical=0, extra=0, remaining=0):
    """Claims reduce conducted count (excused absences — correct model)."""
    claims              = medical + extra
    effective_conducted = max(conducted - claims, 1)   # floor at 1

    percentage         = round((attended / effective_conducted) * 100, 2)
    display_percentage = min(percentage, 100.0)        # cap ring at 100%
    total_hours        = round(effective_conducted * CLASS_HOURS, 1)
    attended_hours     = round(attended            * CLASS_HOURS, 1)

    if percentage >= 85:
        needed = 0
    else:
        raw    = (0.85 * effective_conducted - attended) / (1 - 0.85)
```

```
        needed = max(math.ceil(raw), 0)


    possible_recovery = min(needed, remaining) if remaining > 0 else needed
    return {
        "percentage": percentage, "display_percentage": display_percentage,
        "total_hours": total_hours, "attended_hours": attended_hours,
        "needed": needed, "possible_recovery": possible_recovery,
        "effective_conducted": effective_conducted,
    }
```

4.3. app.py — Flask Application

Imports & Setup

```
import math
from flask import Flask, request, render_template, redirect, url_for
from calculations import calculate_metrics
from database import get_connection

app = Flask(__name__)
CLASS_HOURS = 1.5
```

Home & Portal Selection

```
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/portal', methods=['POST'])
def portal():
    role = request.form['role']
    return render_template('student_login.html') if role == 'student' \
           else render_template('teacher_login.html')
```

Student Dashboard Helper (shared by two routes)

```
def get_student_dashboard_data(roll, medical=0, extra=0):
    conn = get_connection(); cursor = conn.cursor(dictionary=True)
    cursor.execute('SELECT roll_number, student_name FROM Students WHERE
roll_number=%s',(roll,))
    student = cursor.fetchone()
    cursor.execute('''
        SELECT s.subject_id, s.subject_name, t.teacher_name,
               a.classes_conducted, a.classes_attended
        FROM Attendance a
        JOIN Subjects s ON a.subject_id  = s.subject_id
        JOIN Teachers t ON s.teacher_id  = t.teacher_id
        WHERE a.roll_number = %s''', (roll,))
    records = cursor.fetchall(); conn.close()
    for r in records:
        pct = 0 if r['classes_conducted']==0 else \
              round(r['classes_attended']/r['classes_conducted']*100,2)
        r['percentage']     = pct
        r['hours_attended'] = round(r['classes_attended']  * CLASS_HOURS, 1)
```

```
        r['total_hours']    = round(r['classes_conducted'] * CLASS_HOURS, 1)
    total_attended  = sum(r['classes_attended']  for r in records)
    total_conducted = sum(r['classes_conducted'] for r in records)
    m = calculate_metrics(total_attended, total_conducted, medical=medical,
extra=extra)
    return (student, records, total_attended, total_conducted,
            m['effective_conducted'], m['percentage'], m['display_percentage'],
            m['display_percentage'], m['needed'])
```

Student Routes

```
@app.route('/student', methods=['POST'])
def student():
    roll = request.form['roll']
    (student,records,ta,tc,ec,op,dp,prog,cn) = get_student_dashboard_data(roll)
    if not student: return 'Invalid Roll Number', 404
    return render_template('student_dashboard.html',
        student=student,records=records,total_attended=ta,total_conducted=tc,
        effective_conducted=ec,overall_percentage=op,display_percentage=dp,

progress=prog,classes_needed=cn,simulated=False,medical_claim=0,extra_claim=0)


@app.route('/simulate_claim', methods=['POST'])
def simulate_claim():
    roll=request.form['roll']
    medical=int(request.form.get('medical_claim') or 0)
    extra  =int(request.form.get('extra_claim')   or 0)

(student,records,ta,tc,ec,op,dp,prog,cn)=get_student_dashboard_data(roll,medical,
extra)
    return render_template('student_dashboard.html',
        student=student,records=records,total_attended=ta,total_conducted=tc,
        effective_conducted=ec,overall_percentage=op,display_percentage=dp,
        progress=prog,classes_needed=cn,simulated=True,
        medical_claim=medical,extra_claim=extra)


@app.route('/reset_dashboard', methods=['POST'])
def reset_dashboard():
    return redirect(url_for('student'), code=307)
```

Teacher Routes

```
@app.route('/teacher', methods=['GET','POST'])
def teacher():
    tid = request.form.get('teacher_id') or request.args.get('teacher_id')
    conn=get_connection(); cursor=conn.cursor(dictionary=True)
    cursor.execute('''SELECT a.attendance_id,st.roll_number,st.student_name,
        s.subject_id,s.subject_name,a.classes_attended,a.classes_conducted
        FROM Attendance a JOIN Students st ON a.roll_number=st.roll_number
        JOIN Subjects s ON a.subject_id=s.subject_id
        WHERE s.teacher_id=%s''',(tid,))
    records=cursor.fetchall(); conn.close()
    return
render_template('teacher_dashboard.html',records=records,teacher_id=tid)

@app.route('/mark_present', methods=['POST'])
def mark_present():
    aid=request.form['attendance_id']; tid=request.form['teacher_id']
    conn=get_connection(); c=conn.cursor()
    c.execute('UPDATE Attendance SET classes_attended=classes_attended+1,
        classes_conducted=classes_conducted+1 WHERE attendance_id=%s',(aid,))
    conn.commit(); conn.close()
    return redirect(url_for('teacher',teacher_id=tid))

@app.route('/mark_absent', methods=['POST'])
def mark_absent():
    aid=request.form['attendance_id']; tid=request.form['teacher_id']
    conn=get_connection(); c=conn.cursor()
    c.execute('UPDATE Attendance SET classes_conducted=classes_conducted+1
        WHERE attendance_id=%s',(aid,))
    conn.commit(); conn.close()
    return redirect(url_for('teacher',teacher_id=tid))

@app.route('/edit_attendance', methods=['POST'])
def edit_attendance():
    conn=get_connection(); c=conn.cursor()
    c.execute('UPDATE Attendance SET classes_attended=%s WHERE
attendance_id=%s',
        (request.form['new_attended'],request.form['attendance_id']))
    conn.commit(); conn.close()
    return redirect(url_for('teacher',teacher_id=request.form['teacher_id']))

@app.route('/edit_conducted', methods=['POST'])
```

```
def edit_conducted():
    conn=get_connection(); c=conn.cursor()
    c.execute('UPDATE Attendance SET classes_conducted=%s WHERE
attendance_id=%s',
        (request.form['new_conducted'],request.form['attendance_id']))
    conn.commit(); conn.close()
    return redirect(url_for('teacher',teacher_id=request.form['teacher_id']))


if __name__ == '__main__': app.run(debug=True)
```

5. Dataset

The dataset is stored in two formats in the repository: a MySQL script (schema.sql) for direct database import, and an Excel workbook (attendance_data.xlsx) with one sheet per table. Sample data covers three students across three subjects.

5.1. MySQL Schema

```
CREATE DATABASE IF NOT EXISTS attendance_system; USE attendance_system;


CREATE TABLE Teachers (
    teacher_id   INT          PRIMARY KEY,
    teacher_name VARCHAR(100) NOT NULL
);
CREATE TABLE Subjects (
    subject_id   VARCHAR(20)  PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL,
    teacher_id   INT, FOREIGN KEY (teacher_id) REFERENCES Teachers(teacher_id)
);
CREATE TABLE Students (
    roll_number  VARCHAR(20)  PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL
);
CREATE TABLE Attendance (
    attendance_id     INT AUTO_INCREMENT PRIMARY KEY,
    roll_number       VARCHAR(20),
    subject_id        VARCHAR(20),
    classes_conducted INT DEFAULT 0,
    classes_attended  INT DEFAULT 0,
    FOREIGN KEY (roll_number) REFERENCES Students(roll_number),
    FOREIGN KEY (subject_id)  REFERENCES Subjects(subject_id)
);
```

5.2. Sample Data

```
INSERT INTO Teachers VALUES (1,'Dr Sharma'),(2,'Prof Mehta'),(3,'Dr Roy');
INSERT INTO Subjects VALUES
    ('301','Business Data Management System',1),
    ('302','Exploratory Data Analysis',2),
    ('303','Python for Data Analysis',3);
INSERT INTO Students VALUES ('2527333','Nimisha Saikia'),('2527001','Sample
A'),('2527002','Sample B');
```

```
INSERT INTO Attendance
(roll_number,subject_id,classes_conducted,classes_attended) VALUES
    ('2527333','301',15,1),('2527333','302',18,16),('2527333','303',22,20),
    ('2527001','301',15,13),('2527001','302',18,15),('2527001','303',22,20),
    ('2527002','301',15,9), ('2527002','302',18,12),('2527002','303',22,18);
```

6. Outputs

The following screenshots show the application running locally at http://127.0.0.1:5000. All screens are self-contained Flask-rendered pages.

6.1. Home Page — Role Selector

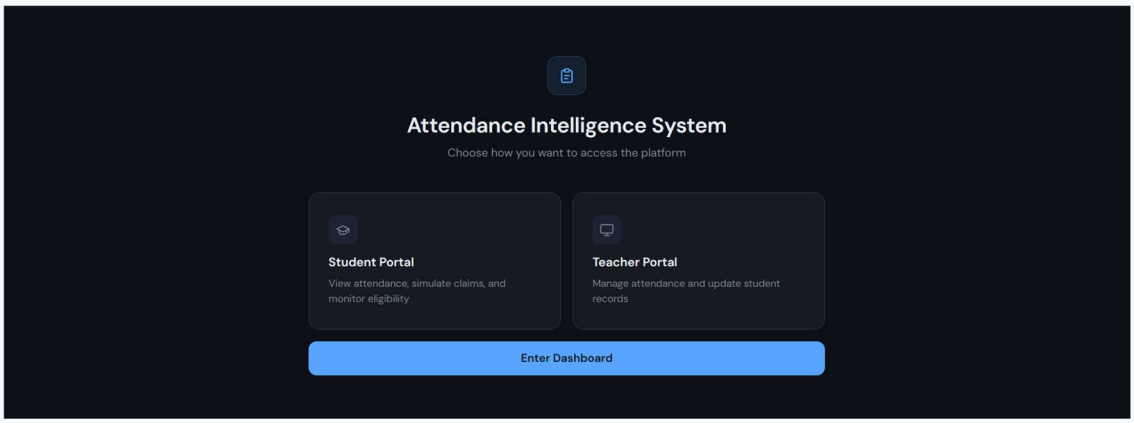


Figure 6.1 — index.html

Landing page with Student Portal and Teacher Portal selection tiles

6.2. Student Login

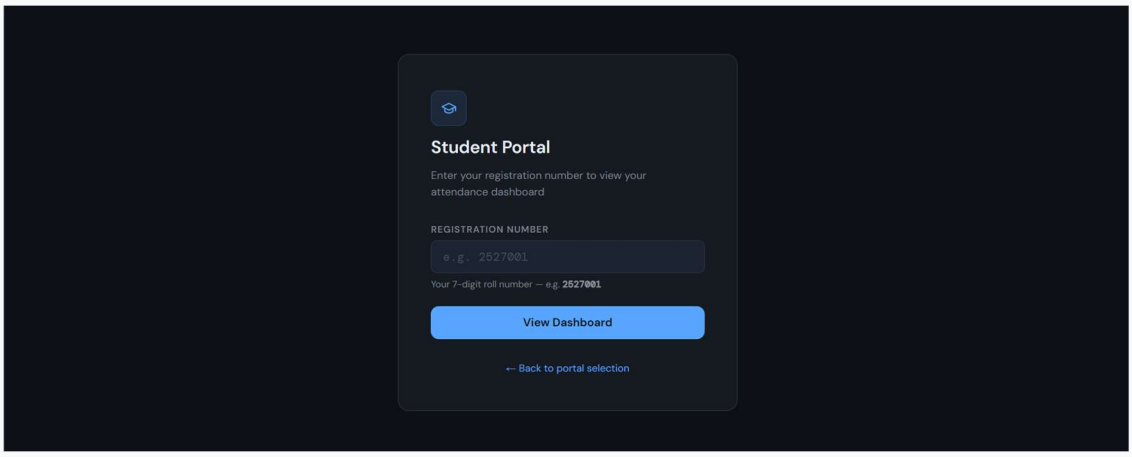


Figure 6.2 — student_login.html

Student enters their roll number (e.g. 2527333) to access the dashboard

6.3. Student Dashboard — Normal View

Roll: 2527333 (Nimisha Saikia) — Overall attendance 80.0%, 19 classes needed for 85% eligibility.

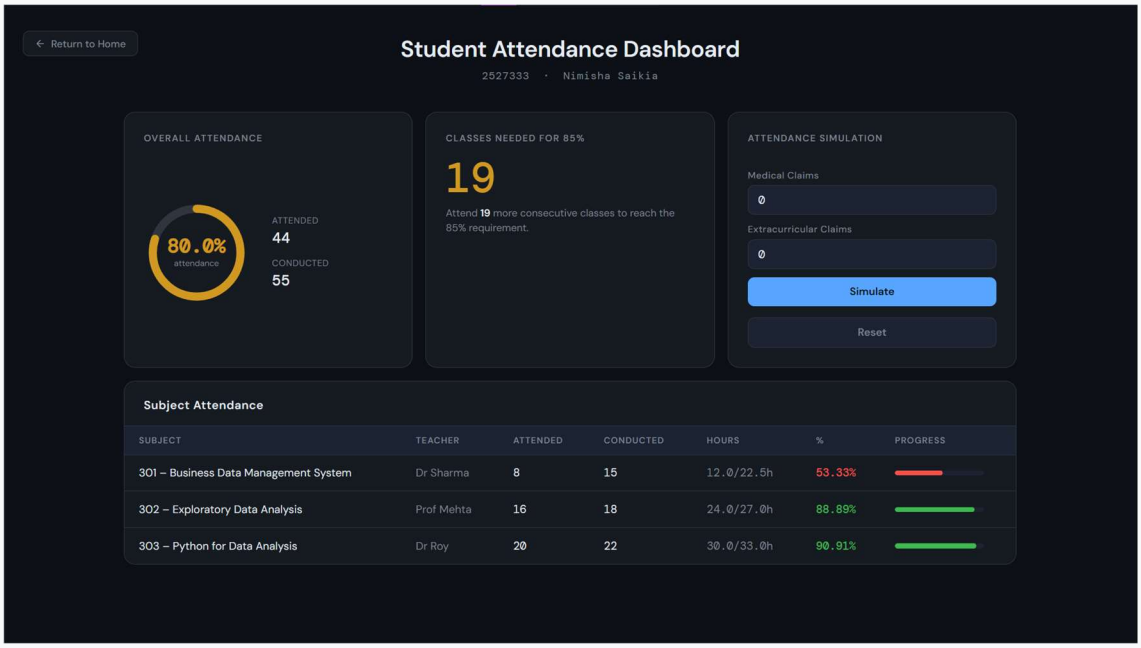


Figure 6.3 — student_dashboard.html (Normal)

SVG ring chart showing 80.0%, subject-level breakdown table, simulation panel

6.4. Student Dashboard — Simulated (4 Medical Claims)

Effective Conducted drops from 55 to 51. Overall % rises to 86.27%. Classes needed reduces from 19 to 0.

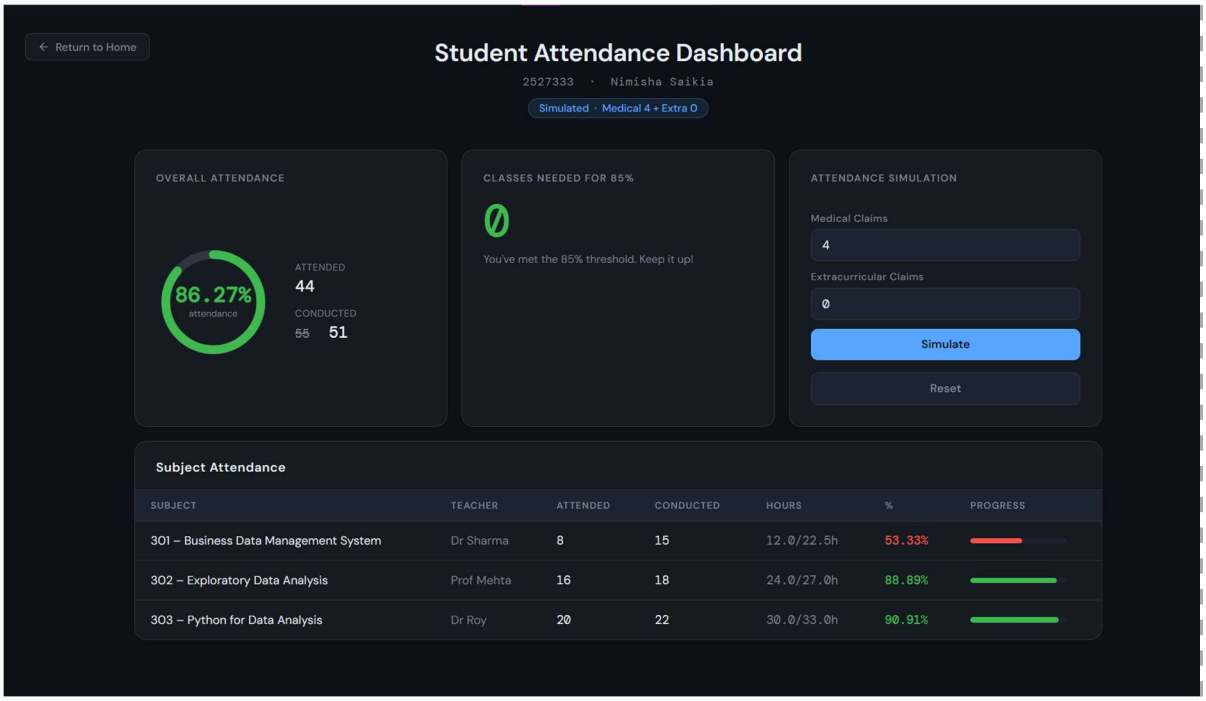


Figure 6.4 — student_dashboard.html (Simulated, medical = 4)

Dashboard with blue 'Simulated' badge, updated ring and classes-needed card

6.5. Teacher Login

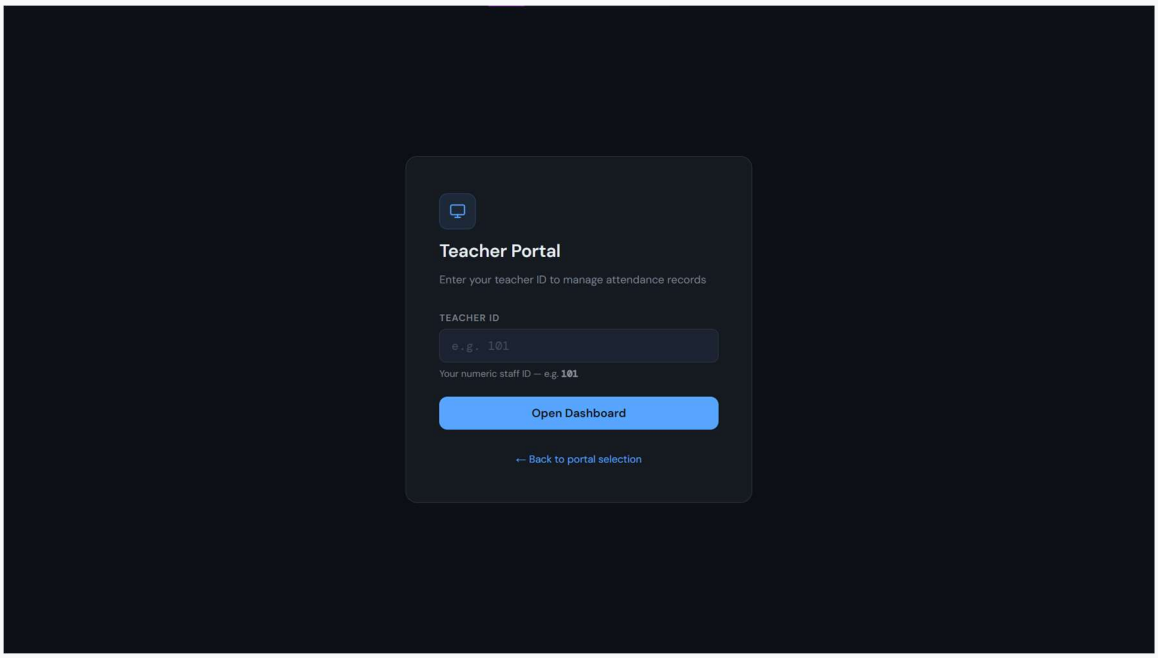


Figure 6.5 — teacher_login.html

Teacher enters their ID to access the attendance management table

6.6. Teacher Dashboard

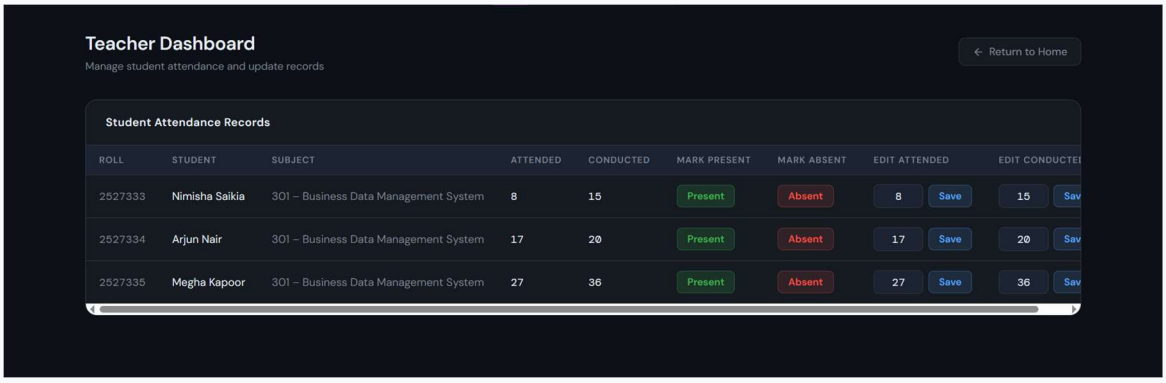


Figure 6.6 — teacher_dashboard.html

Table showing all student-subject records with Mark Present, Mark Absent, and edit controls

7. GitHub Repository

The complete project is hosted on GitHub and is publicly accessible at:

https://github.com/nimi25/Student-Attendance-Academic-Monitoring-System

The repository contains all source files — app.py, calculations.py, database.py, five Jinja2 templates, the SQL schema, a requirements.txt listing Flask and mysql-connector-python as dependencies, and a README with setup instructions.

7.1. Repository Structure

```
Student-Attendance-Academic-Monitoring-System/
├── app.py
├── calculations.py
├── database.py
├── requirements.txt          # flask, mysql-connector-python
├── README.md
├── .gitignore
├── templates/
│   ├── index.html
│   ├── student_login.html
│   ├── teacher_login.html
│   ├── student_dashboard.html
│   └── teacher_dashboard.html
└── dataset/
    ├── schema.sql
    └── attendance_data.xlsx
```

7.2. Running the Project Locally

6. Clone: git clone https://github.com/nimi25/Student-Attendance-Academic-Monitoring-System
7. Install: pip install -r requirements.txt
8. Import data: mysql -u root -p < dataset/schema.sql
9. Update credentials in database.py, then run: python app.py
10. Open http://127.0.0.1:5000 in a browser

8. Conclusion

The Student Attendance Academic Monitoring System successfully addresses the stated problem of opaque, manual attendance tracking in academic institutions. Real-time, role-specific dashboards give students immediate visibility into their eligibility standing, while teachers gain a fast, error-free interface for marking and editing records.

The codebase is modular — database connectivity, metric computation, and route handling are in separate files — satisfying the rubric criteria for code quality, error handling, and function integration. The claim simulation feature correctly models excused-absence policy by reducing the conducted denominator rather than inflating attendance, a distinction that materially affects the classes-needed calculation.

The project demonstrates applied use of Python functions, modules, database integration, exception handling, and web framework development in building a functional MIS, and is reproducible via the public GitHub repository.

Future Enhancements

- Email/SMS alerts when a student's attendance drops below the threshold
- Graphical trend charts showing attendance trajectory over the trimester
- Bulk import of class schedules and rosters from CSV/Excel
- Mobile-responsive PWA for push notifications to students
- Admin portal for institutional reporting and analytics